Lecture -2

P, NP, NP-Complete, and NP-Hard Problems

By Dr. Gopal Chandra Jana, Assistant Professor, DCSE, SSCSE

https://www.gcjana.in/courses/shardauniversity/2502/PHD650/

Recap

What is Algorithm Analysis?

Algorithm analysis studies:

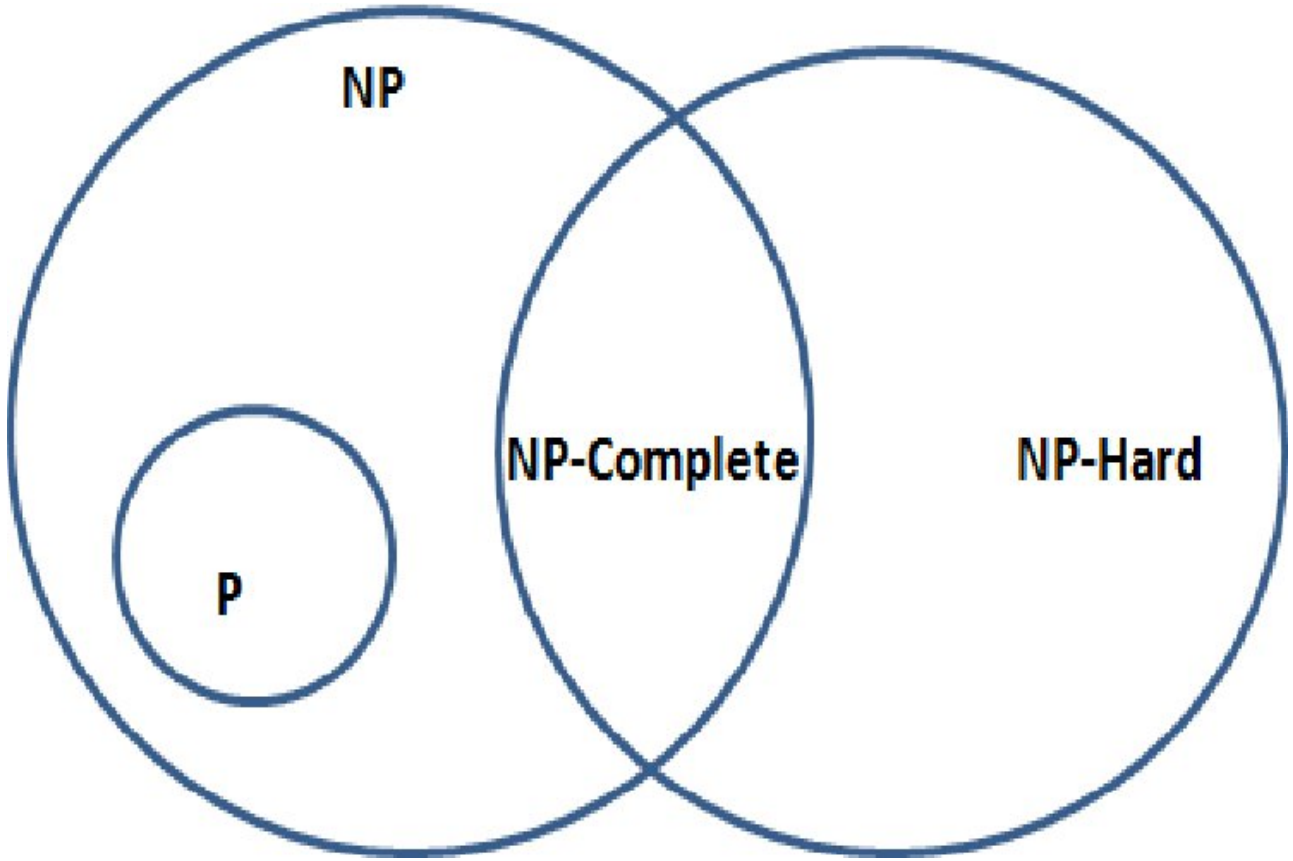
- **Time Complexity** – How running time grows with input size
- **Space Complexity** – Memory usage
- **Worst Case**, **Average Case**, **Best Case**
- Asymptotic Notations:
  - **Big-O (O)** – Upper bound
  - **Theta (Θ)** – Tight bound
  - **Omega (Ω)** – Lower bound

Complexity Classes

A. Class **P** (Polynomial Time - deterministic)

B. Class **NP** (Non-deterministic Polynomial Time)

- ❖ NP-Complete Problems
- ❖ NP-Hard Problems



Complexity Classes - Class P (Polynomial Time - deterministic)

A problem belongs to **P** if it can be solved in polynomial time:

$$T(n) = O(n^k)$$ for some constant k.

Complexity Classes - Class P (Polynomial Time - deterministic)

What is Deterministic?

A computation is **deterministic** if:

- At every step, the machine has **exactly one possible next move**.
- There is **no guessing**.
- For a given input → the execution path is fixed.

Complexity Classes - Class P (Polynomial Time - deterministic)

What is Deterministic?

Problem: Search an element in a **sorted array**.

```
BinarySearch(A, n, key):
    low = 0
    high = n-1
    while low ≤ high:
        mid = floor((low + high)/2)
        if A[mid] == key:
            return mid
        else if key < A[mid]:
            high = mid - 1
        else:
            low = mid + 1
    return -1
```

mid = (low + high) / 2

Or

mid = low + (high - low) / 2

Both are mathematically equivalent, **but ?**

Complexity Classes - Class P (Polynomial Time - deterministic)

What is Deterministic?

Problem: Search an element in a **sorted array**.

```
BinarySearch(A, n, key):
    low = 0
    high = n-1
    while low ≤ high:
        mid = floor((low + high)/2)
        if A[mid] == key:
            return mid
        else if key < A[mid]:
            high = mid - 1
        else:
            low = mid + 1
    return -1
```

mid = (low + high) / 2 Or mid = low + (high - low) / 2

Both are mathematically equivalent, **but not computationally equivalent due to integer overflow.**

Complexity Classes - Class P (Polynomial Time - deterministic)

What is Deterministic?

Problem: Search an element in a **sorted array**.

mid = (low + high) / 2 Or mid = low + (high - low) / 2

In most systems:

- ❖ int is **32-bit signed**
- ❖ Range: −2^31 to 2^31−1

  −2,147,483,648 to 2,147,483,647

low = 2,000,000,000

high = 2,100,000,000

So maximum integer:

INT_MAX = 2,147,483,647

Low + high = 4,100,000,000

low + high > INT_MAX

This exceeds INT_MAX.

Overflow happens BEFORE division

Complexity Classes - Class P (Polynomial Time - deterministic)

What is Deterministic?

Problem: Search an element in a **sorted array**.

mid = (low + high) / 2 Or mid = low + (high - low) / 2

low = 2,000,000,000

high = 2,100,000,000

So maximum integer:

INT_MAX = 2,147,483,647

Low + high = 4,100,000,000

low + high > INT_MAX

This exceeds INT_MAX.

Overflow happens BEFORE division

4,100,000,000−2^32

= 4,100,000,000−4,294,967,296

= −194,967,296

mid = - 194,967,296 / 2

= −97,483,648

high − low= 2,100,000,000 − 2,000,000,000 = 100,000,000/2=50,000,000

mid = 2,000,000,000 + 50,000,000 =2,050,000,000

```
BinarySearch(A, n, key):
    low = 0
    high = n - 1

    while low ≤ high:
        mid = low + (high - low) // 2

        if A[mid] == key:
            return mid

        else if key < A[mid]:
            high = mid - 1

        else:
            low = mid + 1

    return -1
```

Is it Deterministic?

Binary Search complexity

n = number of elements

Binary search works on a sorted array

and at each step:

- ❖ It compares with middle element.
- ❖ It reduces the problem size to **half**.

So, after one step:
New size = n/2

Binary Search complexity

n = number of elements

Binary search works on a sorted array

and at each step:

- It compares with middle element.
- It reduces the problem size to **half**.

So, after one step:
New size = n/2

Form the Recurrence Relation:

T(n) = number of comparisons in worst case

At each step:

1. One comparison (constant work → 1)
2. Recursive call on half size

So, recurrence becomes:

$$\mathbf{T(n) = T(n/2) + 1}$$

T(n/2) = T(n/4) + 1 => **T(n) = T(n/4) + 2**

T(n/4)=T(n/8)+1 => **T(n) = T(n/8) + 3**

After **k** expansions: T(n) = **T(n/2^k) + k**

n/2^k = 1, Recursion stops when problem size becomes 1. Now take log of both sides

Binary Search complexity

Step	Size
0	n
1	n/2
2	n/4
3	n/8
k	$n/2^k$

T(n) = O(log n)

$$n \rightarrow n/2 \rightarrow n/4 \rightarrow n/8 \rightarrow \cdots \rightarrow 1$$

```
NonDet-TwoSum(A, target):
    Guess two indices i and j
    If A[i] + A[j] == target:
        Accept
    Else:
        Reject
```

Is it Deterministic?

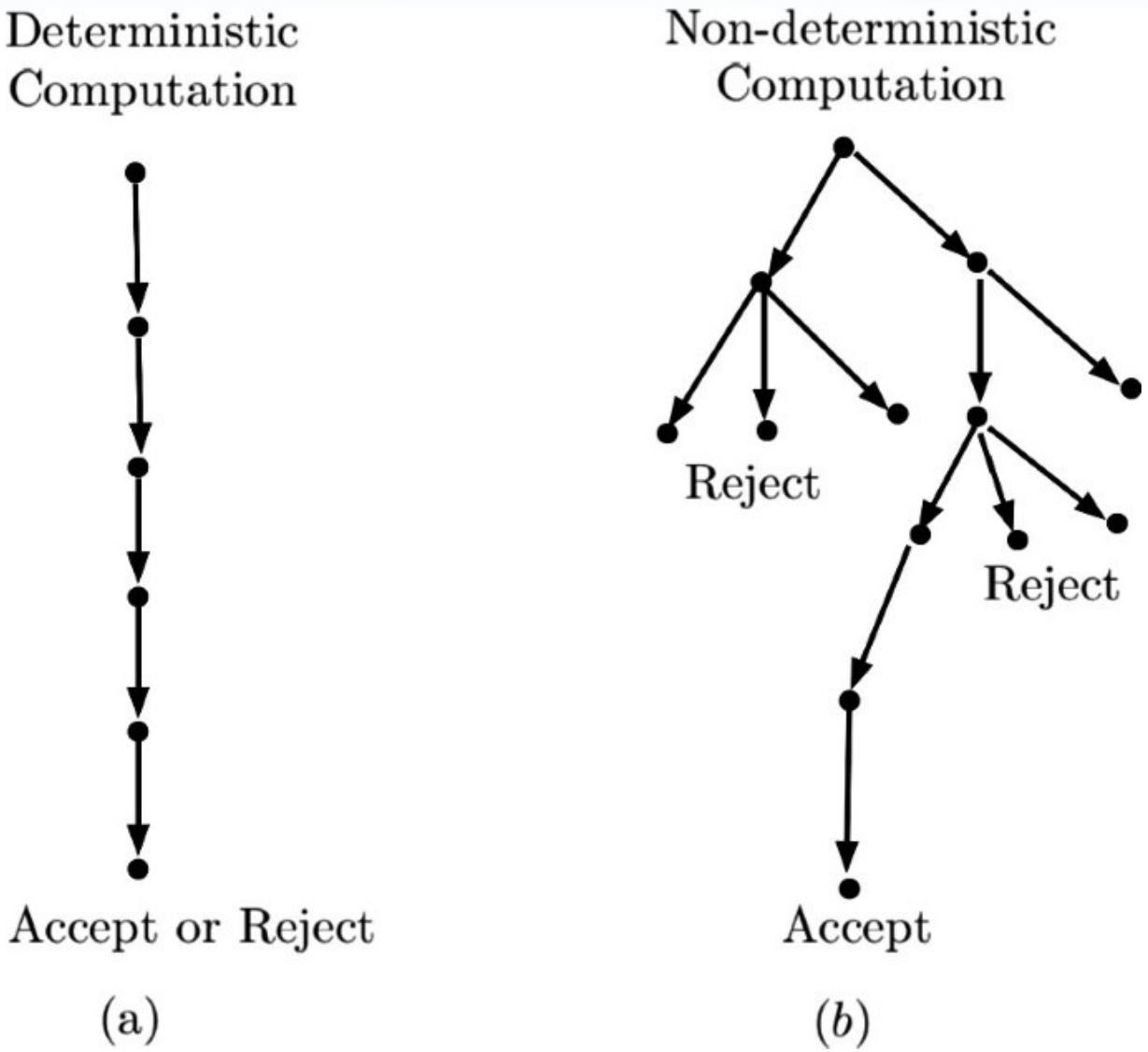
```
BFS(G, s):
    Mark all vertices unvisited
    Create empty queue Q
    Mark s visited
    Enqueue s

    while Q not empty:
        u = Dequeue(Q)
        for each neighbor v of u:
            if v not visited:
                Mark v visited
                Enqueue v
```

Is it Deterministic?

```
DFS(G, u):
    Mark u visited
    for each neighbor v of u:
        if v not visited:
            DFS(G, v)
```

Is it Deterministic?


Deterministic Computation
Accept or Reject
(a)
Non-deterministic Computation
Reject
Reject
Accept
(b)

Subset Sum (Decision Version)

Given set $A = \{a_1, a_2, ..., a_n\}$ and integer K,
is there a subset whose sum equals K?

```
SubsetSum(A, n, K):
    for each subset S of A:
        if sum(S) == K:
            return TRUE
    return FALSE
```

Is it in P or NP?

Subset Sum (Decision Version)

```
SubsetSum(A, n, K):
    for each subset S of A:
        if sum(S) == K:
            return TRUE
    return FALSE
```

Is it in P or NP?

How many subsets? -> **2^n** (an element can be include or exclude)

To compute sum(S): **O(n)**

Total time: T(n) = 2^n×n = O(n 2^n) -> **O(2^n)**

Subset Sum (Decision Version)

```
SubsetSum(A, n, K):
    for each subset S of A:
        if sum(S) == K:
            return TRUE
    return FALSE
```

How many subsets? -> **2^n** (an element can be include or exclude)

To compute sum(S): **O(n)**

Total time: T(n) = 2^n×n = O(n 2^n) -> **O(2^n)**

Is it in NP.

✔ Exponential ✔ Not polynomial

So deterministic brute-force solution is NOT polynomial time.

Subset Sum (Decision Version)

```
SubsetSum(A, n, K):
    for each subset S of A:
        if sum(S) == K:
            return TRUE
    return FALSE
```

Is it in NP or NP Hard, or NP Complete?

How many subsets? -> **2^n** (an element can be include or exclude)

To compute sum(S): **O(n)**

Total time: T(n) = 2^n×n = O(n 2^n) -> **O(2^n)**

Subset Sum (Decision Version)

```
SubsetSum(A, n, K):
    for each subset S of A:
        if sum(S) == K:
            return TRUE
    return FALSE
```

Is it in NP or NP Hard, or NP Complete?

How many subsets? -> **2^n** (an element can be include or exclude)

To compute sum(S): **O(n)**

Total time: T(n) = 2^n×n = O(n 2^n) -> **O(2^n)**

NP does NOT mean:

"Can be solved in polynomial time"

NP means:

"If someone gives you a candidate solution, you can **verify it in polynomial time**."

$$\text{NP-Complete} = \text{NP} \cap \text{NP-Hard}$$

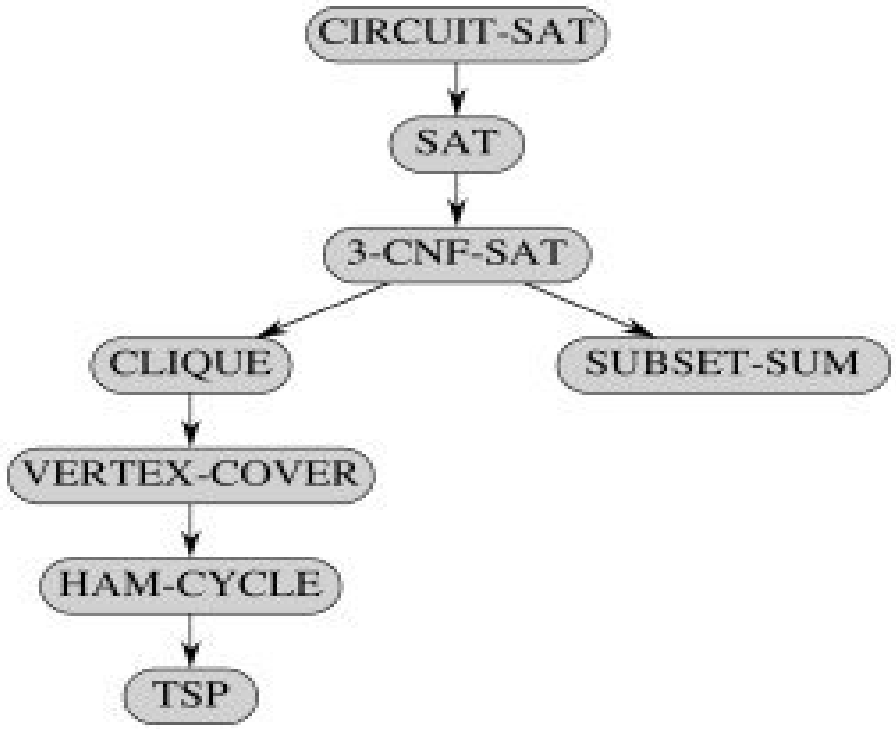
Suppose someone gives you: S={3,5} and claims:

sum(S)=K

To **verify**:

- Compute sum(S)

- Compare with K

This takes: **O(n)**

Concept	Meaning
Polynomial solving	$\in$ P
Polynomial verification	$\in$ NP
Both NP + NP-Hard	NP-Complete